OBJECT-ORIENTED PROGRAMMING

CL-1004

LAB 05

has-a Relationship, and Array of Objects

National University of Computer and Emerging Sciences–NUCES – Karachi

Instructor: Talha Shahid

Contents

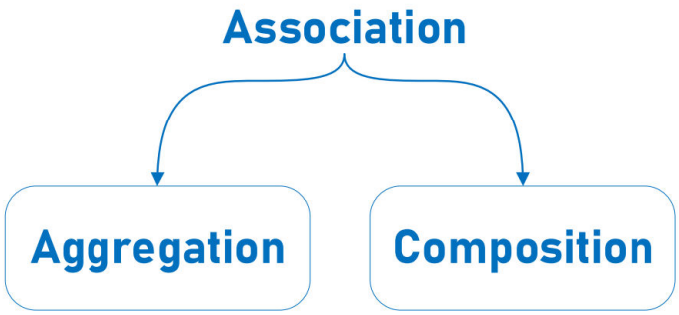
4. has-a Relationship

4.1. Association

A "**has-a**" relationship, also known as **association**, represents a connection between two classes where one class contains or owns another class as a member. In this relationship, one class has another class as a member, typically through a pointer or reference. The class that contains the other class is referred to as the owner or container, while the class that is contained is referred to as the member or component.

In simpler terms, a "**has-a**" relationship implies that one class possesses or is associated with another class. This association can be **one-to-one**, **one-to-many**, or **many-to-many**. The containing class provides access to the contained class's functionality and may manipulate or interact with it in various ways.

For example, a `car` "**has-a**" engine, a `department` "**has-a**" manager, or a `library` "**has-a**" collection of books. These relationships help model real-world scenarios in object-oriented programming and enable building more complex systems by composing simpler components.



4.1.1. Aggregation

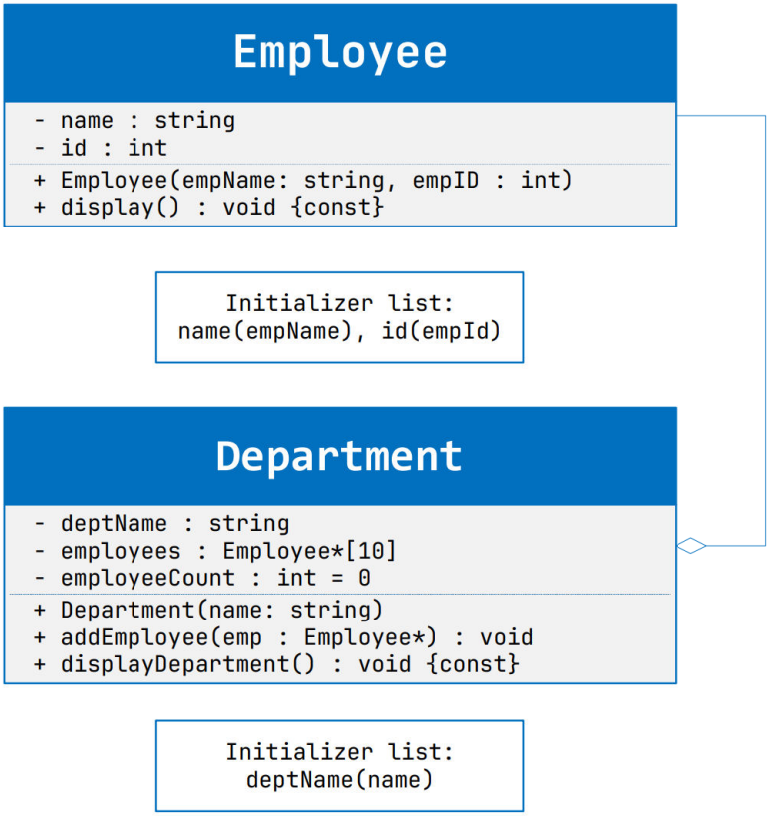
Aggregation is a specialized form of association where objects have a "**whole-part**" relationship, but the parts can exist independently of the whole. The whole may contain parts, but the parts are not dependent on the existence of the whole. It represents a weaker form of ownership. **For example**, a `Department` class can have multiple `Employee` objects, but the employees can exist independently of the department.

Example 01

```
#include <iostream>
using namespace std;

class Employee {
public:
    string name;
    int id;
```

```
    Employee(string empName, int empId) : name(empName), id(empId) {}

    void display() const {
        cout << "Employee ID: " << id << ", Name: " << name << endl;
    }
};

class Department {
private:
    string deptName;
    Employee* employees[10];
    int employeeCount = 0;

public:
    Department(string name) : deptName(name) {}

    void addEmployee(Employee* emp) {
        employees[employeeCount++] = emp;
    }

    void displayDepartment() const {
        cout << "Department: " << deptName << endl << "Employees:" << endl;
        for (int i = 0; i < employeeCount; i++) employees[i]->display();
    }
};

int main() {
    Employee e1("Shaheen Shah Afridi", 10), e2("Muhammad Rizwan", 16), e3("Babar
Azam", 56);
    Department dept("Software Engineering");

    dept.addEmployee(&e1);
    dept.addEmployee(&e2);
    dept.displayDepartment();

    cout << endl << "Independent Employee:" << endl;
    e3.display();

    return 0;
}
```

Output	Department: Software Engineering Employees: Employee ID: 10, Name: Shaheen Shah Afridi Employee ID: 16, Name: Muhammad Rizwan Independent Employee: Employee ID: 56, Name: Babar Azam



4.1.2. Composition

Composition is a stronger form of association where the parts are tightly coupled to the whole. The parts are created and destroyed along with the whole. If the whole is destroyed, all its parts are destroyed as well. It represents a stronger form of ownership.

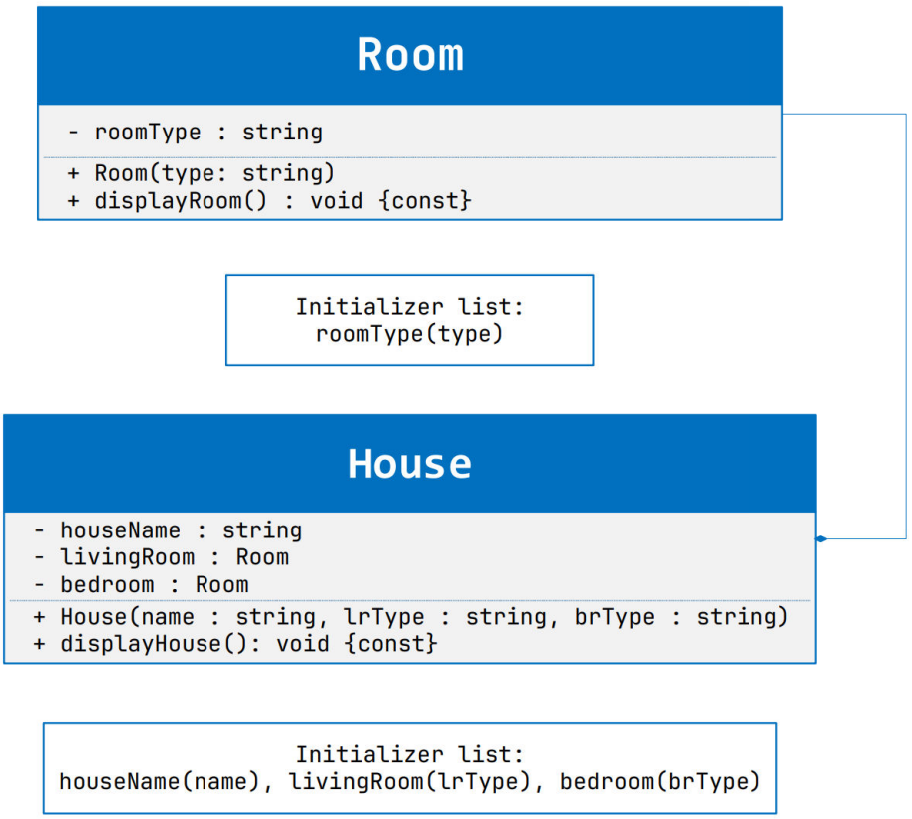
For example, a `House` object might contain `Rooms`, and when the `House` is destroyed, all its `Rooms` are also destroyed.

Example 01:

```cpp
#include <iostream>
using namespace std;

class Room {
private:
    string roomType;

public:
    Room(string type) : roomType(type) {}

    void displayRoom() const {
        cout << "Room Type: " << roomType << endl;
    }
};

class House {
private:
    string houseName;
```

```
    Room livingRoom;
    Room bedroom;

public:
    House(string name, string lrType, string brType)
        : houseName(name), livingRoom(lrType), bedroom(brType) {}

    void displayHouse() const {
        cout << "House Name: " << houseName << endl;
        livingRoom.displayRoom();
        bedroom.displayRoom();
    }
};

int main() {

    House myHouse("Dream Villa", "Living Room", "Master Bedroom");
    myHouse.displayHouse();

}
```

Output	House Name: Dream Villa Room Type: Living Room Room Type: Master Bedroom



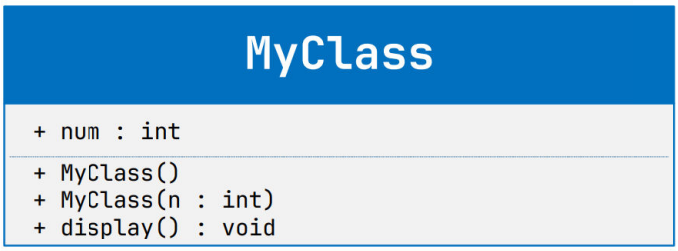
5. Array of Objects

In C++, you can create **arrays of objects** just like you create arrays of **primitive data types**. Arrays of objects allow you to store **multiple objects** of the **same class** in contiguous memory locations.

Example 01:

```
#include "iostream"
using namespace std;

class MyClass{
public:
    int num;

    MyClass(){

        num = 0;
    }

    MyClass(int n){

        num = n;
    }

    void display() {

        cout << "Number: " << num << endl;
    }
};

int main(){

    const int size = 5;
    MyClass arr[size]; // array of MyClass objects

    // initializing objects in the array
    for (int i = 0; i < size; i++){

        arr[i] = MyClass(i+1); // initializing with numbers 1 through 5
    }

    // displaying objects in the array
    for (int i = 0; i < size; i++){

        arr[i].display(); // displaying the number of each object
    }

}
```

Output	Number: 1 Number: 2 Number: 3 Number: 4 Number: 5



In this example:

- We define a class `MyClass` with a member variable `num`.
- There are two constructors: one default constructor which initializes `num` to `0`, and another constructor that takes an integer parameter and sets `num` to the value of the parameter.
- The `display()` method prints the value of `num` for an object.
- In the `main()` function, we create an array `arr` of `MyClass` objects with a size of `5` (size).
- We initialize each object in the array with a number from `1` to `5` using a for loop.
- Finally, we iterate through the array and call the `display()` method for each object to print its number.
- This demonstrates how you can use arrays of objects in C++ to manage multiple objects of the same class efficiently.

Example 02:

```
#include <iostream>
#include <cstring>
using namespace std;

class Employee{

    int id;
    char name[30];
public:

    void take_data(int id, char name[]){
        this->id = id;
        strcpy(this->name, name); // using strcpy() to copy name
    };

    void display_data(){
        cout << id << " " << name << " " << endl;
    };
};


int main(){

    Employee emp[30];
    int n, i;
```

```
    cout << "Enter number of employees: ";
    cin >> n;

    for (int i = 0; i < n; i++){

        int id;
        char name[30];

        cout << "Enter Id: "; cin >> id;
        cout << "Enter Name: "; cin >> name;

        emp[i].take_data(id, name);
    }

    cout << "Employee Data" << endl;

    for (int i = 0; i < n; i++){

        emp[i].display_data();
    }

}
```

Output	Enter number of employees: 5 Enter Id: 1 Enter Name: ABC Enter Id: 2 Enter Name: EFG Enter Id: 3 Enter Name: HIJ Enter Id: 4 Enter Name: KLM Enter Id: 5 Enter Name: NOP Employee Data 1 ABC 2 EFG 3 HIJ 4 KLM 5 NOP